HW3

> I affirm that I personally wrote the text, code, and comments in this homework assignment.
>
> I received help from [other student] who gave me suggestions on [problem].

- [Aashna Sibal]

PageRank

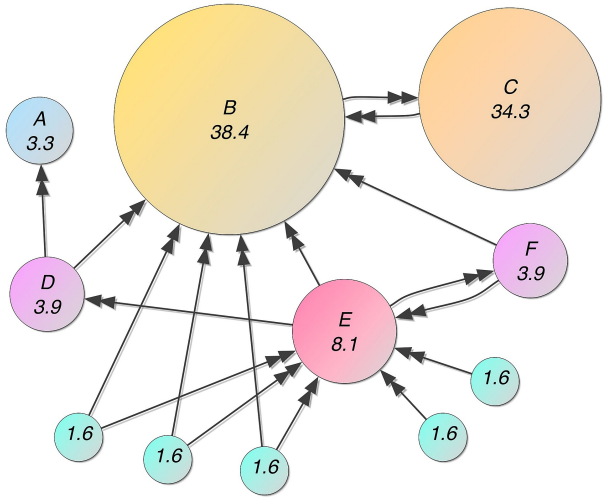
What is the most important website on the internet? Who is the "key player" on a sports team? Which countries are the most central players in the world economy? There is no one correct answer to any of these questions, but there is a most profitable one. PageRank is an algorithm for ranking individual elements of complex systems, invited by Sergey Brin and Larry Page. It was the first and most famous algorithm used by the Google Search engine, and it is fair to say that the internet as we know it today would not exist without PageRank.

In this assignment, we will implement PageRank. There are many good ways to implement this algorithm, but in this assignment we will use our newfound skills with object-oriented programming and iterators.

Comments are required for all parts of this assignment

How it works

For the purposes of this example, let's assume that we are talking about webpages. PageRank works by allowing a "random surfer" to move around webpages by following links. Each time the surfer lands on a page, it then looks for all the links on that page. It then picks one at random and follows it, thereby arriving at the next page, where the process repeats. Eventually, the surfer will visit all the pages one or more times. Pages that the surfer visits more frequently have higher *PageRank scores*. Because the surfer moves between linked pages, PageRank expresses an intuitive idea: **important pages are linked to other important pages.** This diagram from Wikipedia gives a nice illustration. Note that more important webpages (higher PageRank) tend to be connected to other important webpages.



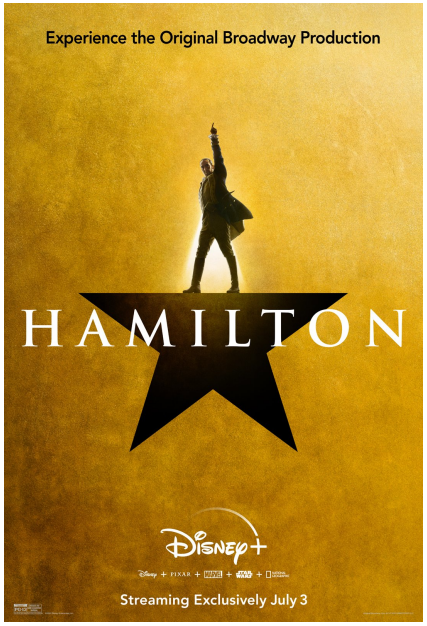
A schematic for PageRank.

Data

You can complete this assignment using data from one of two sources.

Option 1: Hamilton

This data set comes from the hit Broadway musical "Hamilton."

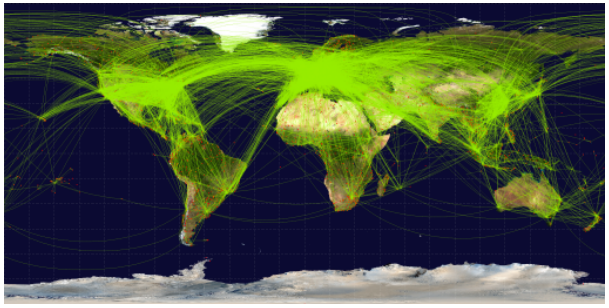


The Hamilton data set

The good folks at The Hamilton Project analyzed the script for us, obtaining data on **who talks about whom** in each of the show's songs. When character A mentions character B, we'll think of this as a *link* from A to B, suggesting that B might be important.

If you use this data set, listening to the soundtrack while working is strongly recommended.

Option 2: Global Airline Network



The global airline network

Back in the Before Times, lots of people flew on airplanes. This data set includes a "link" from Airport A to Airport B whenever there is a flight from B to A. This data set was collected by the OpenFlights Project.

(A). Define Functions

In this part, all you have to do is hit `shift + enter` on the code block supplied below. This block defines two functions. The first one retrieves the data from the internet and saves it to your local computer, while the second reads in the data, producing a list of tuples. It's not important for you to be familiar with the code in these functions right now -- we'll discuss them soon.

In [1]:
```
import urllib
import csv

def retrieve_data(url):
    """
    Retrieve a file from the specified url and save it in a local file
    called data.csv. The intended values of url are:

    1. https://philchodrow.github.io/PIC16A/homework/HW3-hamilton-data.csv
    2. https://philchodrow.github.io/PIC16A/homework/HW3-flights-data.csv
    """

    # grab the data and parse it
    filedata = urllib.request.urlopen(url)
    to_write = filedata.read()

    # write to file
    with open("data.csv", "wb") as f:
        f.write(to_write)

def read_data(path):
    """
    read downloaded data from a .csv file, and return a list of tuples.
    each tuple represents a link between states.
    """
    with open(path, "r") as f:
        reader = csv.reader(f)
        return [(row[0], row[1]) for row in list(reader)]
```

(B). Grab the Data

The data live at the following URLs:

- **Hamilton**: `https://philchodrow.github.io/PIC16A/homework/HW3-hamilton-data.csv`
- **Airline**: `https://philchodrow.github.io/PIC16A/homework/HW3-flights-data.csv`

In each data set, each row corresponds to a "link" between objects. In Hamilton, the pairs have format `mentioner, mentioned` while in the airline network the rows have format `destination, origin.`

Pick one of these data sets, and set the variable `url` appropriately by uncommenting one of the two lines below. Then, call `retrieve_data()` and `read_data()`. The `path` argument for `read_data()` should be set to `"data.csv"`. Create a variable called `data` to hold the return value of `read_data()`.

Your solution

In [2]:
```
# uncomment the second line if you'd prefer to
# work with the flights data.
url = "https://philchodrow.github.io/PIC16A/homework/HW3-hamilton-data.csv"
# url = "https://philchodrow.github.io/PIC16A/homework/HW3-flights-data.csv"

# Call your functions below
retrieve_data(url)                # retrieving the data from the url
data = read_data("data.csv")      # calling the function to read the data
```

(C). Examine the structure of the data

This would also be a good time to inspect the data to make sure you understand how it is structured. Write a function `describe(n)` that describes the meaning of the `n` th row of the data set you chose. In the Hamilton data set, your function should do the following:

```
describe(5)

# output
"Element 5 of the Hamilton data set is ('burr', 'betsy'). This means that Burr mentions Betsy in
a song."
```

In context of the airline flights data, your function should instead do this:

```
describe(5)
```

```
# output
"Element 5 of the flights data set is ('SIN', 'BKK'). This means that there is a flight from BKK
to SIN."
```

Please attend to capitalization and formatting. While the standard string concatenation operator `+` is completely fine for this task, the fancy `str.format()` function may make your code somewhat simpler. This page has some useful examples in case you'd like to try this.

Your Solution

In [3]:
```
def describe(n):
    """A function that describes the meaning of the nth row of a data set.

    parameter 1, n: the row of the dataset whose meaning the function will describe
    """
    tup1 = data[n]                          #assigning each tuple in the data list to a variable so we can
                                            #index through
    mentioner = str(tup1[0]).capitalize() #the first tuple index is the mentioner, converting it to a
                                            #capitalized string for formatting purposes
    mentioned = str(tup1[1]).capitalize() #the second tuple index is the mentioned, converting it to a
                                            #capitalized string for formatting purposes
    print("Element "+str(n)+" of the Hamilton data set is "+str(tup1)+". This means that "+mentioner+ \
          " mentions "+mentioned+" in a song.")  #printing the statement with the appropriate
                                                 #mentioner/mentioned

describe(5)                                 #calling the function to describe the 5th row of the dataset
```

```
Element 5 of the Hamilton data set is ('burr', 'betsy'). This means that Burr mentions Betsy in a song.
```

(D). Data to Dictionary

Write a function called `data_to_dictionary` that converts the data into a dictionary such that:

1. There is a single key for each character (in Hamilton) or airport (in flights).
2. The value corresponding to each key is a list of the characters/airports to which that key links. The list should contain repeats if there are multiple links.

Here's an example of the desired behavior on a fake data set.

```
data = [("a", "b"),
        ("a", "b"),
```

```
        ("a", "c"),
        ("b", "c"),
        ("b", "a")]

data_to_dictionary(data)

# output
{"a" : ["b", "b", "c"], "b" : ["a", "c"]}
```

Your Solution

In [4]:

```
def data_to_dictionary(data):
    """
    A function that converts the data from the data set into a dictionary where there is a single key for
    each character and the value corresponding to each key is a list of the characters to which that key
    links. This includes repeats in the list for multiple links.

    parameter 1, data: the data set used to make the dictionary
    return, D: the dictionary with the corresponding keys and values
    """
    D = {}                              # initializing an empty dictionary

    for n in range(len(data)):          # to index through each tuple in the data list of tuples
        tup = data[n]                   # assigning each index of data to a "tup" variable so we can index
                                        # through
        if tup[0] in D:                 # if the mentioner is already in D
            D[tup[0]].append(tup[1])    # add the corresponding mentioned to the value of that key in D
        else:                           # if the mentioner is NOT in D yet,
            D[tup[0]] = [tup[1]]        # add a new key of the mentioner with the value of mentioned to D

    return D                            # return the completed dictionary

data_to_dictionary(data)                # calling the function to create D from the data
```

Out[4]:

```
{'burr': ['hamilton',
  'weeks',
  'madison',
  'jay',
  'theodosiaDaughter',
  'betsy',
  'theodosiaMother',
  'hamilton',
  'hamilton',
  'hamilton',
  'washington',
  'hamilton',
  'marthaWashington',
  'schuylerSis',
  'washington',
  'burr',
  'generalMontgomery',
  'hamilton',
  'philipS',
  'peggy',
  'angelica',
  'eliza',
  'hamilton',
  'reynolds',
  'hamilton',
  'washington',
  'hamilton',
  'philipS',
  'generalMercer',
  'madison',
  'jefferson',
  'washington',
  'hamilton',
  'washington',
  'jefferson',
  'jefferson',
  'madison',
  'burr',
  'hamilton',
  'hamilton',
  'jAdams',
  'jefferson',
  'hamilton',
  'jefferson',
  'burr',
```

```
 'ness',
 'hamilton',
 'pendleton',
 'angelica',
 'eliza'],
'hamilton': ['burr',
 'angelica',
 'philipH',
 'lafayette',
 'eliza',
 'laurens',
 'mulligan',
 'washington',
 'eliza',
 'lee',
 'laurens',
 'conway',
 'hamilton',
 'washington',
 'lee',
 'laurens',
 'burr',
 'washington',
 'hamilton',
 'burr',
 'lee',
 'burr',
 'eliza',
 'peggy',
 'angelica',
 'hamilton',
 'laurens',
 'mulligan',
 'lafayette',
 'burr',
 'kingGeorge',
 'burr',
 'lafayette',
 'laurens',
 'burr',
 'hamilton',
 'reynolds',
 'eliza',
 'angelica',
 'philipH',
```

```
 'eliza',
 'eacker',
 'philipH',
 'eliza',
 'reynolds',
 'jefferson',
 'madison',
 'burr',
 'reynolds',
 'washington',
 'jefferson',
 'washington',
 'kingLouis',
 'lafayette',
 'burr',
 'burr',
 'angelica',
 'maria',
 'reynolds',
 'angelica',
 'madison',
 'jefferson',
 'eliza',
 'schuylerSis',
 'jAdams',
 'jefferson',
 'washington',
 'madison',
 'jefferson',
 'hamilton',
 'philipH',
 'eliza',
 'burr',
 'jefferson',
 'jAdams',
 'burr',
 'hamilton',
 'burr',
 'laurens',
 'washington',
 'eliza'],
'ensemble': ['washington',
 'kingGeorge',
 'jefferson',
 'burr',
```

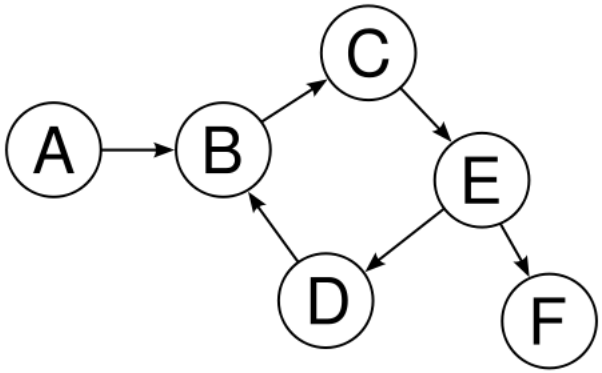
```
 'hamilton',
 'jAdams',
 'jefferson'],
'company': ['hamilton',
 'mulligan',
 'lafayette',
 'hamilton',
 'washington',
 'hamilton',
 'admiralHowe',
 'washington',
 'kingGeorge',
 'schuylerSis',
 'angelica',
 'reynolds',
 'washington',
 'jefferson',
 'hamilton',
 'burr',
 'jefferson',
 'eliza',
 'jAdams',
 'burr'],
'men': ['hamilton', 'angelica', 'jAdams', 'jefferson', 'burr'],
'women': ['hamilton', 'angelica', 'washington', 'eliza', 'burr', 'jefferson'],
'angelica': ['hamilton',
 'hamilton',
 'angelica',
 'franklin',
 'schuylerSis',
 'eliza',
 'angelica',
 'eliza',
 'burr',
 'paine',
 'jefferson',
 'schuylerSis',
 'hamilton',
 'jefferson',
 'angelica',
 'eliza',
 'angelica',
 'hamilton',
 'eliza',
 'angelica',
```

```
 'eliza'],
'eliza': ['hamilton',
 'washington',
 'hamilton',
 'eliza',
 'eliza',
 'eliza',
 'angelica',
 'schuylerSis',
 'angelica',
 'eliza',
 'hamilton',
 'hamilton',
 'philipH',
 'angelica',
 'jAdams',
 'angelica',
 'washington',
 'hamilton',
 'hamilton'],
'washington': ['rochambeau',
 'hamilton',
 'burr',
 'lee',
 'hamilton',
 'hamilton',
 'lee',
 'lafayette',
 'hamilton',
 'burr',
 'green',
 'knox',
 'jefferson',
 'jefferson',
 'hamilton',
 'burr',
 'hamilton',
 'jefferson',
 'madison',
 'jefferson'],
'mulligan': ['mulligan', 'hamilton', 'burr', 'mulligan', 'burr'],
'lafayette': ['hamilton', 'hamilton', 'burr', 'lafayette'],
'laurens': ['hamilton',
 'lee',
 'burr',
```

```
 'angelica',
 'laurens',
 'sAdams',
 'burr'],
'kingGeorge': ['washington', 'jAdams'],
'jefferson': ['hamilton',
 'reynolds',
 'eliza',
 'hamilton',
 'washington',
 'hamilton',
 'washington',
 'lafayette',
 'hamilton',
 'washington',
 'madison',
 'burr',
 'hamilton',
 'lafayette',
 'washington',
 'sally',
 'madison',
 'jAdams',
 'hamilton',
 'burr',
 'washington',
 'hamilton'],
'madison': ['hamilton',
 'washington',
 'hamilton',
 'hamilton',
 'burr',
 'jefferson',
 'hamilton',
 'burr',
 'jefferson',
 'hamilton',
 'jAdams'],
'philipH': ['eacker', 'philipH', 'philipS', 'burr', 'philipH'],
'lee': ['lee', 'washington'],
'peggy': ['peggy', 'schuylerSis'],
'seabury': ['seabury', 'kingGeorge'],
'reynolds': ['reynolds'],
'doctor': ['hamilton']}
```

(E). Define a PR_DiGraph class

A **directed graph**, or DiGraph, is just a set of arrows ("edges") between objects ("nodes"). It is a natural way to represent data that represents one-way relationships, such as links from one webpage to another or mentions of one character by another. We already saw a directed graph above when we introduced the idea of PageRank. Here's a paired-down example.



Example of a directed graph.

Implement a `PR_DiGraph` class with a custom `__init__()` method and a `linked_by()` method. The `__init__()` method should accept two arguments: `data` and `iteration_limit`. The `__init__()` method should then construct an instance variable `self.link_dict` which is simply the output of `data_to_dictionary` applied to the argument `data`. `__init__()` should also construct an instance variable `self.iteration_limit`, which simply takes the value of `iteration_limit` supplied to `__init__()`. Don't worry about that one for now.

Then, define a method `self.linked_by(x)` which, when called, returns the value `self.link_dict[x]`.

Finally, add an `__iter__` method, which returns an object of class `PR_Iterator`. We will define this class in the next part.

Example session (using Hamilton):

```
D = PR_DiGraph(data, iteration_limit = 10000)
D.linked_by('peggy')

# output
['peggy', 'schuylerSis']
```

Your Solution

In [5]:
```
class PR_DiGraph:

    def __init__(self,data,iteration_limit):
        """
        Constructor that is called when an object is created from the class.
        Helps initialize the attributes of the class.
        """
        self.data = data                        # creating instance variables for unique objects of the class
        self.iteration_limit = iteration_limit
        self.link_dict = data_to_dictionary(data) # constructing an instance variable that is the output of
                                                  # data_to_dictionary when applied to the argument data

    def linked_by(self,x):
        """
        Returns the value of self.link_dict, or the output of data_to_dictionary(data)
        """
        self.x = x                              # creating instance variable for unique objects of the class
        return(self.link_dict[x])

    def __iter__(self):
        """
        Returns an object of class PR_Iterator
        """
        return(PR_Iterator(self))

D = PR_DiGraph(data, iteration_limit = 10000)
D.linked_by('peggy')                            # seeing who peggy is linked to in the list/who she mentions
```

Out[5]:
```
['peggy', 'schuylerSis']
```

(F). Implement PR_Iterator

Define a `PR_Iterator` class with a custom `__next__()` method.

The `__init__` method of this class should create instance variables to store the `PR_DiGraph` object from which it was constructed; a counter `i`, starting at 0, to log the number of steps taken, and a `current_state` variable whose value is one of the keys of the `link_dict` of the `Pr_DiGraph`. You can choose its initial value arbitrarily; in my solution code I chose `self.current_state = "hamilton"`.

We are going to use iteration to implement the PageRank algorithm. This means we are going to imagine a surfer who is following the links in our data set. **Implement the following two methods:**

1. `follow_link()`.
    A. Pick a random new character mentioned by the current character, or new airport served by the current airport. Let's call this `next_state`.
    B. If `next_state != current_state`, set `current_state` to `next_state`.
    C. Otherwise (if `next_state == current_state`), teleport (see below).
    D. You might run into `KeyError`s, in which case you should again teleport (use a `try-except` block).
2. `teleport()`.
    A. Set the current state to a new state (key of the link dict) completely at random.

Hint: use `random.choice` from the `random` module to choose elements of lists.

Finally, **implement** `__next__()`. `__next__()` should do `follow_link()` with 85% probability, and do `teleport()` with 15% probability. You should also define a custom `StopIteration` condition to ensure that only as many steps are taken as the `iteration_limit` supplied to the `PR_DiGraph` initializer.

1. To do something with 85% probability, use the following:

```
if random.random() < 0.85:
    # do the thing
else:
    # do the other thing
```

Example Usage

After you define your class, run the following code and show that it works. Note: your precise sequence may be different from mine.

```
D = PR_DiGraph(data, iteration_limit = 5)
for char in D:
    print(char)
```

```
following link : current state = burr
following link : current state = washington
following link : current state = burr
following link : current state = hamilton
teleporting    : current state = washington
```

I have added printed messages here for you to more clearly see what should be happening, but it is not necessary for you to do this. It is sufficient for your output to look like:

```
D = PR_DiGraph(data, iteration_limit = 5)
for char in D:
    print(char)

```

```
burr
washington
burr
hamilton
washington
```

Your Solution

In [6]:

```
import random
class PR_Iterator:
    def __init__(self,D):
        """
        Constructor that is called when an object is created from the class.
        Helps initialize the attributes of the class.
        """
        self.D = D
        self.i = 0                        # starting the counter at 0
        self.current_state = "hamilton" # setting the current_state arbitrarily

    def follow_link(self):
        """
        Picks a random new character mentioned by the current character. If the new character is not
        equal to the current one, set the current one to the new one. Otherwise if they are the same,
        teleport to a new character. If you run into KeyErrors, teleport again.
        """
        try:                                                                    # try this
            next_state = random.choice(self.D.linked_by(self.current_state)) # randomly picking new character
                                                                                # to be the next_state
            if next_state != self.current_state:                                # if current and next not same
                self.current_state = next_state                                 # make current state new state
            else:                                                               # if states are the same,
                self.teleport()                                                 # set current to new random one
        except KeyError:                                                        # when you encounter a KeyError
            self.teleport()                                                     # teleport to a new character

    def teleport(self):
```

```
        """
        Sets the current character to a new one mentioned by the current completely at random.
        """
        self.current_state = random.choice(list(self.D.link_dict.keys())) # randomly picks a character
                                                                          # mentioned by the current one
                                                                          # and assigns it to current state


    def __next__(self):
        """
        Does the follow_link() function with 85% probability and teleport() with 15% probability.
        """
        if random.random() < 0.85:              # 85% probability
            self.follow_link()                  # do follow_link() function
            self.i+=1                           # increasing counter by 1
        else:
            self.teleport()                     # in 15% probability, teleport to a new mentioned character
            self.i+=1                           # increasing counter by 1

        if self.i > self.D.iteration_limit:     # if counter/number of iterations is greater than the limit
            raise StopIteration                 # stop iteration
        return self.current_state
```

In [7]:
```
# run the below
D = PR_DiGraph(data, iteration_limit = 5)
for char in D:
    print(char)
```

```
jefferson
hamilton
mulligan
company
burr
```

(G). Compute PageRank

Finally, we are ready to compute the PageRank in our data set. Initialize a `PR_DiGraph` with a large iteration limit (say, 1,000,000). Use a `for` -loop to allow your surfer to randomly move through the data set. The number of times that the surfer visits state `x` is the PageRank score of `x` .

Create a `dict` which logs how many times a given state appears when iterating through the `PR_Digraph` . So, this dictionary holds the PageRank score of each state.

Your Solution

In [8]:
```
X = PR_DiGraph(data, iteration_limit = 1000000)
D = {}                # initializing an empty dictionary

for i in X:           # indexing through the PR_DiGraph
    if(i not in D):   # if a character is not in the dictionary
        D[i] = 0      # add it to D with value 0 (assigning the unique characters to keys in D)
    else:             # each time the character is present in the PR_DiGraph,
        D[i] += 1     # increase their key's value (or PageRank score) by 1 in the dictionary
```

(H). Display Your Result

Use your favorite approach to show the results in sorted format, descending by PageRank score. The entries at the top should be the entries with highest PageRank. What are the most important elements in the data set?

You may show either the complete list or just the top 10.

Check your code by comparing your top 10 to mine. Because we are using a randomized algorithm, your results will not agree exactly with mine, but they should be relatively close. If your top 10 list is very different, then you might want to revisit your previous solutions.

For Hamilton, my top 10 were:

```
[('hamilton', 166062),
 ('burr', 99180),
 ('washington', 92246),
 ('jefferson', 72450),
 ('eliza', 51485),
 ('angelica', 48042),
 ('madison', 37421),
 ('lafayette', 34297),
 ('lee', 33678),
 ('jAdams', 31121)]
```

For the flights data, my top 10 were:

```
[('LHR', 18043), # London Heathrow
 ('ATL', 16370), # Atlanta
 ('JFK', 14795), # New York JFK
 ('FRA', 14156), # Frankfurt
 ('CDG', 14073), # Charles de Gaulle (Paris)
 ('LAX', 13199), # Los Angeles
 ('ORD', 12915), # Chicago O'Hare
 ('PEK', 12525), # Beijing
 ('AMS', 12410), # Amsterdam Schiphol
 ('PVG', 11517)] # Shanghai
```

Your solution

In [9]:
```
sorted(D.items(), key=lambda item: item[1], reverse=True)

#printing the dictionary's items, but sorting them by their values (or PageRank score)
#reverse=True so that it is sorted in descending order rather than ascending
```

Out[9]:
```
[('hamilton', 165340),
 ('burr', 99075),
 ('washington', 92699),
 ('jefferson', 72623),
 ('eliza', 51726),
 ('angelica', 47998),
 ('madison', 36959),
 ('lafayette', 34272),
 ('lee', 33719),
 ('jAdams', 31229),
 ('reynolds', 29152),
 ('kingGeorge', 29034),
 ('laurens', 27496),
 ('philipH', 26431),
 ('mulligan', 21534),
 ('peggy', 20653),
 ('schuylerSis', 19164),
 ('ensemble', 17173),
 ('seabury', 17129),
 ('men', 17013),
 ('company', 17011),
 ('doctor', 16998),
 ('women', 16997),
 ('philipS', 7850),
 ('eacker', 6243),
 ('rochambeau', 4093),
 ('knox', 4008),
 ('green', 3962),
 ('sAdams', 3300),
 ('sally', 2801),
 ('franklin', 1893),
 ('paine', 1879),
 ('conway', 1771),
 ('betsy', 1745),
 ('kingLouis', 1733),
 ('jay', 1689),
 ('maria', 1689),
 ('generalMontgomery', 1673),
 ('ness', 1663),
 ('theodosiaDaughter', 1661),
 ('theodosiaMother', 1661),
 ('generalMercer', 1648),
 ('pendleton', 1636),
 ('marthaWashington', 1621),
```

```
('weeks', 1575),
('admiralHowe', 735)]
```

(I). Submit!

Check that your code is appropriately documented (comments and docstrings), and turn it in.